

Napisati Matlab program koji učitava monohromatsku sliku 'slika1.jpg' i obavlja linearno filtriranje slike sa maskom dimenzija 5x1 definisanom tako da računa srednju vrednost piksela ispod maske. Zatim nad dobijenim rezultatom filtriranja izvršiti segmentaciju sa fiksnim pragom $T=10$. Prikazati originalnu sliku, kao i sliku nakon segmentacije.

Resenje:

```
f = imread('slika1.jpg');
if size(f,3) == 3
    rgb2gray(f);
f = double(f);
maska = ones(5,1) / 5;
slika_filtrirana = (f, maska, 'replicate');
T = 10;
slika_segmentisana = slika_filtrirana > T;
imshow('slika1.jpg');
subplot(1, 2, 1);
title("Originalna_slika.");
imshow('slika_segmentisana');
subplot(1,2,2);
title("Segmentisana_slika.");
```

1. `f = imread('slika1.jpg');` ✓ – ovo je tačno, učitavaš sliku.
2. `if size(f,3) == 3 rgb2gray(f);` ✗ – ovde problem: funkcija `rgb2gray(f)` vraća novu sliku, ali ti je nisi dodelio promenljivoj. Trebalo bi:

```
if size(f,3) == 3
    f = rgb2gray(f);
end
```

3. `f = double(f);` ✓ – konvertuješ u double, tačno.
4. `maska = ones(5,1) / 5;` ✓ – maska 5x1 za srednju vrednost piksela je ok.
5. `slika_filtrirana = (f, maska, 'replicate');` ✗ – ovde fali funkcija, treba `imfilter`:

```
slika_filtrirana = imfilter(f, maska, 'replicate');
```

6. `T = 10; slika_segmentisana = slika_filtrirana > T;` ✓ – segmentacija je u redu.

`imshow` se koristi za promenljive, ne za imena fajlova u ovom slučaju (osim ako želiš da ponovo učitáš). Treba pise `imshow (f)`;

Kada prebacujem sliku iz RGB u grayscale, ne smem da zaboravim smestanje u promenljivu `f`. Znaci ja trebam da napisem uslov ovaj, `if size (f, 3) == 3`, to da, jer je `size` velicina, prva trojka se odnosi na 3. dimenziju matrice `f`, a druga ova trojka proverava da li ta dimenzija ima vrednost 3, odnosno da li je slika RGB. Ali ne mozemo samo da napisemo `rgb2gray(f);`, vec to mora da se smesti u promenljivu `f`. Prebacivanje u podatak tipa `double` je takodje obavezno, jer ako se to ne uradi onda nece moci da se barata kako treba u smislu da nece moci da se izvrsavaju aritmeticke operacije nad pikselima.

Kada se spominje linearno filtriranje, ja treba da se setim maske `one` u kojoj su sve jedinice. E sad, sve te jedinice treba podeliti sa dimenzijom maske. Na primer, u ovom konkretnom slucaju, maska je dimenzija 5 x 1, i to je 5 elemenata. Znaci matricu koja se oznacava sa `ones` delimo sa 5. Da je matrica dimenzija 5 x 2, onda bi delili sa 10, itd. Znaci tako ide postupak. Konkretno za ovaj zadatak, u Matlabu kazemo: `maska = ones ( 5, 1) / 5`.

Napisati Matlab program koji učitava monohromatsku sliku 'slika1.jpg' i obavlja linearno filtriranje slike sa maskom dimenzija 5×1 definisanom tako da računa srednju vrednost piksela ispod maske. Zatim nad dobijenim rezultatom filtriranja izvršiti segmentaciju sa fiksnim pragom $T=10$. Prikazati originalnu sliku, kao i sliku nakon segmentacije.

Resenje:

```
f = imread('slika1.jpg');
if size(f,3) == 3
    rgb2gray(f);
f = double(f);
maska = ones(5,1) / 5;
slika_filtrirana = (f, maska, 'replicate');
T = 10;
slika_segmentisana = slika_filtrirana > T;
imshow('slika1.jpg');
subplot(1, 2, 1);
title("Originalna_slika.");
imshow('slika_segmentisana');
subplot(1,2,2);
title("Segmentisana_slika.");
```

1. `f = imread('slika1.jpg');` ✓ – ovo je tačno, učitavaš sliku.
2. `if size(f,3) == 3 rgb2gray(f);` ✗ – ovde problem: funkcija `rgb2gray(f)` vraća novu sliku, ali ti je nisi dodelio promenljivoj. Trebalo bi:

```
if size(f,3) == 3
    f = rgb2gray(f);
end
```

3. `f = double(f);` ✓ – konvertuješ u double, tačno.
4. `maska = ones(5,1) / 5;` ✓ – maska 5×1 za srednju vrednost piksela je ok.
5. `slika_filtrirana = (f, maska, 'replicate');` ✗ – ovde fali funkcija, treba `imfilter`:

```
slika_filtrirana = imfilter(f, maska, 'replicate');
```

6. `T = 10; slika_segmentisana = slika_filtrirana > T;` ✓ – segmentacija je u redu.

subplot mora ići pre imshow da bi se slike prikazale u istom prozoru.

Takodje, kada nam se u zadatku spominje linearno filtriranje, obavezno se treba setiti na naredbu `imfilter`. To je naredba koja je ključna. Pre te naredbe, potrebno je učitati sliku, sliku prebaciti u grayscale oblik, prebaciti je u oblik tipa double da bi mogli izvoditi aritmetičke operacije nad tom slikom, definisati masku po vec opisanom postupku, i onda napisati naredbu: `slika_filtrirana = imfilter ( f , maska , 'replicate' );` . I da, obavezno je da nazovemo drugacije, kao sto smo ovde napisali `slika_filtrirana`, a ne mozemo da opet napisemo `f`. Glupo je. A sto se tice naredbe 'replicate' , to je zbog ivice tako, zato da ivice ne bi previse odstupale od susednih piksela.

E sad da pricamo malo o segmentaciji slike. Sto se tice segmentacije, opet moramo da napisemo novi naziv. Ali znaci, mi segmentisemo vec filtriranu sliku, tako da bi trebalo da nazovemo `slika_segmentisana` novu promenljivu, onu u koju smestamo rezultat operacije, a samu operaciju obavljamo nad promenljivom `slika_filtrirana`. I sada, sta je fora. Kazemo $T = 10$, i ispod pisemo naredbu: `slika_segmentisana = slika_filtrirana > T`. To je sve bukvalno.

4. Napisati Matlab program koji učitava kolor (RGB) sliku pod nazivom 'slika4.jpg' i izračunava kontrast samo unutar B kanala slike. Kontrast je definisan kao odnos maksimalnog i minimalnog intenziteta u slici. U zadatku nije dozvoljeno koristiti ugrađene Matlab funkcije `max()` i `min()`.

Resenje:

Ok, ovde je pre svega vazno napomenuti par stvari: prvo i prvo, naredba `f` služi za ucitavanje slike pod nazivom 'slika4.jpg' . To vec smo rekli kako se radi. Znacni, treba da napisemo `f = imread('slika4.jpg');`. `read` -> citanje, `imread` -> ucitavanje. OVO JE MNOGO VAZNO STO CU SADA DA KAZEM: Kada izdvajamo B komponentu, NE SMEMO DA ZABORAVIMO DA NAPISEMO `f`. DAKLE, NEISPRAVNO JE DA KAZEMO NA PRIMER: `B = ( :, :, 3 );` TO NI U LUDILU NE SMEM DA NAPISEM. JEDINO JE ISPRAVNO RECI NA SLEDECI NACIN: `B = f(:, :, 3);` . U ovom redu koda nam stoji 3 jer je B treca komponenta kolor slike. Da nam treba G, napisali bi `G = f(:, :, 2);`. Nakon sto ovo uradimo, potrebno je prebacivanje u tip `double`, dakle u podatak tipa `double`. Znacni, izdvojimo tu komponentu koja nam treba, prebacujemo u tip `double`, i onda pisemo dimenzije i oznacavamo minimalni i maksimalni element matrice. Kada pisemo dimenzije `m` i `n`, to mora da bude u uglastim zagradama, ni u ludilu ne sme da bude u oblim zagradama. Znacni unutar njih, uglastih zagrada, pisemo dimenzije `m` i `n`. A unutar oblih zagrada pisemo `B`. Dakle kombinujemo oba tipa zagrada. Ovako: `[m, n] = size(B);`. A kad pisemo minimalni i maksimalni element matrice, tu koristimo oble zagrade isključivo, tu nema kombinovanja, evo i kako to otprilike izgleda. minimalni_element = `B(1,1);` // maksimalni_element = `B(1, 1);` . Posle ovoga je ocigledno sta ide, ide ugnjezdena for petlja, dakle `for(i = 1:1:m)`, to je za vrste, i `for(j = 1:1:n)` za kolone. To je zbog toga sto je matrica dimenzija `m x n`, pri cemu je `m` broj vrsta, a `n` broj kolona. Kad napisemo for petlje jednu ispod druge, onda pisemo dobro poznati if uslov, pa onda sta ce bude ako je ispunjen, pa `end`, pa onda opet if uslov, pa opet sta ce bude ako je ispunjen, pa opet `end`. I onda opet idu jos dva puta `end` za one 2 for petlje napisane jos na pocetku koda.

To je to sto se tice ovog dela koda. A sad, da objasnimo kontrast. Kontrast je kad se podeli maksimalan i minimalan element. Za prikaz rezultata koristimo naredbu `disp`. Takodje, potrebno je znati i za sta je naredba `num2str` (pretvaranje broja u string). **VAZNO !!!! Kad pisemo naredbu `disp`, MORAMO IMATI U VIDU DA ONA NE PRIMA VISE ARGUMENATA, VEC SAMO JEDAN, ZATO CEMO NAPISATI SADA NEISPRAVNU VERZIJU KODA. URADICEMO TO DA BI VIDELI KAKO SE NE RADI OVO. NE SMEM DA ZASEREM NA ISPITU TAKVU STVAR. NEISPRAVNA VERZIJA KODA:**

```
kontrast = maksimalni_element / minimalni_element;
disp ( "Maksimalni element matrice je: ", num2str(maksimalni_element) );
disp ( " Minimalni element matrice je: ", num2str(minimalni_element) );
disp ( " Kontrast je: ", num2str(kontrast) );
```

Napisati Matlab program koji učitava monohromatsku sliku 'zad4.png' i obavlja filtriranje slike u prostornom domenu. Masku filtra definisati tako da su joj dimenzije 3x3 i da izračunava razliku piksela ispod centralne vrste i centralne kolone maske. Na ekranu prikazati originalnu sliku, i sliku nakon filtriranja.

Resenje:

```
f = imread('zad4.png');
if size(f, 3) == 3
    f = rgb2gray(f);
end
f = double(f);
maska = [ 0, 0, 0 ; -1, 0, 1 ; 0, 0, 0 ];
slika_filtrirana = imfilter(f,maska,'replicate');
subplot(1,2,1);
imshow(f);
title("Originalna slika.");
subplot(1,2,2);
imshow(slika_filtrirana);
title("Filtrirana slika.");
```

VAZNE NAPOMENE: NIKADA NE SME DA SE IZOSTAVI NA PRIMER REC

size I f = . U ovom kodu, greska bi bila napisati:

```
if (f,3) == 3          % ne sme da se zaboravi kljucna rec size ovde.
    rgb2gray(f);       % ne sme da se zaboravi f = na pocetku ovoga.
                        % mora da stoji end ovde
```

% ovo je bio primer neispravnog koda, sace napisemo ispravan:

```
if size(f,3) == 3
    f = rgb2gray(f);
end
```

f = double(f) % ovo je, naravno, obavezno.

maska = [0, 0, 0 ; -1, 0, 1 ; 0, 0, 0]; % e sad, ovde je vazno da

% istaknemo neke stvari. Postoje 3 vrste razlicitih maski. Ne moze
% bilo koja da se koristi. U ovom zadatku treba da se odradi razlika
% piksela ispod centralne vrste i centralne kolone maske. Tada se
% koristi maska gde je srednja vrsta -1 0 1, a prva i treca sve 0.

slika_filtrirana = imfilter(f,maska,'replicate');
% ne sme da se zaboravi kljucna rec imfilter.

% I za kraj, najvaznije je da kazemo da subplot uvek ide prvo, pa
% tek onda imshow(f); i imshow(slika_filtrirana);

Napisati Matlab program koji izračunava minimalni element matrice zadate kao $A = \begin{bmatrix} 8 & 3 & 4 & 1 \\ 5 & 2 & 3 & 7 \\ 3 & 5 & 9 & 1 \end{bmatrix}$, bez korišćenja Matlab funkcije `min()`.

Resenje:

```
A = [8, 3, 4, 1; 5 2 3 7; 3 5 9 1];
[m,n] = size(A);
minimalni_element = A(1,1);
for i = 1:1:m
    for j = 1:1:n
        if ( A(i,j) < minimalni_element )
            minimalni_element = A(i,j);
        end
    end
end
disp(['Minimalni element je:', num2str(minimalni_element)]);
```

Samo jedna mala sitnica: u poslednjem redu, kad koristiš `disp` sa tekstom i brojem, treba da se pretvori broj u string, jer MATLAB ne može da spaja tekst i broj direktno.

Sintaksa `for(i:1:m)` nije ispravna u Matlab-u. Trebalo bi `for i = 1:1:m`. Isto važi i za unutrašnju petlju: `for j = 1:1:n`.

U `if` uslovu koristiš `min`, a to je naziv funkcije u Matlab-u. Trebalo bi da upoređuješ sa promenljivom koju si definisao, tj. `minimalni_element`.

Fali ti `end` za unutrašnju i spoljašnju `for` petlju, i još jedan `end` za `if` uslov.

OPSTA PRAVILA !!!!! :

NI U LUDILU NE SMEM DA NAZOVEM PROMENLJIVU `min`. TO NE SMEM DA URADIM NIKAKO I NIKAD. ZATO STO AKO TAKO NAZOVEM IMA DA ZASEREM ZADATAK SKROZ.

JOS VISE IMA DA NAPRAVIM SRANJE AKO NAPISEM `if ( A(i,j) < min )` --> ovo je totalno sranje.

PROMENLJIVA `min` NE POSTOJI, KRAJ PRICE, NEMA !!! MORA SE NAGLASI DA JE FOR PETLJA U PITANJU !!!!! ZNACI, FOR SE JEDNOSTAVNO MORA DA SE PIŠE OVAKO:

```
for i = 1:1:m
    for j = 1:1:n
% USLOV JE VAZAN, EVO KAKO ON GLASI, MORA ZNAM:

        if ( A(i,j) < minimalni_element )
            minimalni_element = A(i,j);
% Na kraju, ide funkcija disp koja prima samo 1
% argument, i raspored je (, pa [, pa ' . |
```

Napisati Matlab program koji učitava kolor (RGB) sliku pod nazivom 'test.jpg' I detektuje horizontalne ivice samo nad G komponentom slike. Za detekciju horizontalnih ivica definisati odgovarajuću masku dimenzija 5×3. Na ekranu prikazati samo G komponentu slike nakon izdvajanja ivica.

Resenje:

```
f = imread('test.jpg');
G = f(:, :, 2);
G = double(G);
maska = [ -1, -1, -1 ; -1, -1, -1 ; 0, 0, 0 ; 1, 1, 1 ; 1, 1, 1 ];
G_filtrirano = imfilter( G, maska, 'replicate' );
```

```
subplot(1,2,1);
imshow(uint8(G));
title('Originalna G komponenta');
```

```
subplot(1,2,2);
imshow(uint8(G_filtrirano));
title('Horizontalne ivice (G komponenta)');
```

uint8(G) – pretvara tvoju matricu u 8-bitnu celobrojnu sliku (0–255), što odgovara originalnim vrednostima piksela. Tako imshow prikazuje sliku onako kako treba. I da, uvek ide subplot pre ovoga imshow, ali vazno je da kazemo da uint8 se koristi da bi se pravilno prikazala slika. To je veoma vazno.

- 1. Dimenzije maske** – Tvoj `maska` je definisana kao `[ -1, -1, -1, -1, -1 ; 0, 0, 0, 0, 0 ; 1, 1, 1, 1, 1 ]`, što je 3×5, a u zadatku traže 5×3. Dakle, redove i kolone treba zameniti i popuniti tako da bude 5 redova i 3 kolone.
- 2. Funkcija za filtriranje** – Napisao si `G_ivice = f(G, maska, 'replicate');` što nije ispravno. Treba da koristiš `imfilter`, tj. `G_ivice = imfilter(G, maska, 'replicate');`.
- 3. disp sa matricom** – `disp(['Nakon izdvajanja ivica je:', num2str(G_ivice)]);` neće raditi ako je `G_ivice` matrica, jer `num2str` može pretvoriti matricu, ali ovako u kombinaciji sa `disp` u jednom stringu nije elegantno za prikaz slike. Bolje je koristiti `imshow(G_ivice)` za vizualizaciju.

Tvoja slika (ili G komponenta) si prebacila u double zbog aritmetičkih operacija nad pikselima (filtriranje, sabiranje, itd.). Međutim, imshow očekuje da su pikseli slike u tipu uint8 ili double između 0 i 1. Ako direktno pozoveš imshow(G) kada je G tip double i vrednosti nisu između 0 i 1, slika neće pravilno da se prikaže – može biti previše svetla, tamna, ili potpuno pogrešno prikazana. Rešenje: uint8(G) – pretvara tvoju matricu u 8-bitnu celobrojnu sliku (0–255), što odgovara originalnim vrednostima piksela. Tako imshow prikazuje sliku onako kako treba.

4. Napisati Matlab program koji učitava kolor sliku (RGB) pod nazivom 'slika4.jpg' i na osnovu nje formira monohromatsku sliku tj. matricu u kojoj su intenziteti piksela dobijeni kao srednja vrednost R, G i B komponenti kolor slike. Nad dobijenom monohromatskom slikom izračunati njen negativ i prikazati rezultat na ekranu.

Resenje:

```
f = imread('slika4.png');

if size(f, 3) == 3
    f = rgb2gray(f);
end

f_neg = 255 - f;
subplot(1,2,1);
imshow(f); title('Originalna slika');
subplot(1,2,2);
imshow(uint8(f_neg)); title('Negativ slike');
```

1. `f = imread(slika4.jpg);`

✗ Ovde ti fali navodnik oko imena fajla. Ispravno je:

```
f = imread('slika4.jpg');
```

2. `if (f,3) == 3`

✗ Sintaksa je pogrešna. Pravilno treba da bude:

```
if size(f,3) == 3
    f = rgb2gray(f);
end
```

— Ovo proverava da li je slika RGB i konvertuje je u grayscale.

3. Negativ slike:

```
f_neg = 255 - f;
```

✓ Ovo je tačno, pravi negativ slike.

5. Napisati Matalab program koji učitava monohromatsku sliku 'slika5.jpg' i obavlja sledeće operacije: Linearno filtriranje slike maskom dimenzija 5x5 koju treba definisati tako da detektuje vertikalne ivice u slici. Prikazati rezultat na ekranu.

Resenje: Kad se učitava slika uz pomoć naredbe `f = imread('slika5.jpg');` ne smeju da se zaboravljaju znaci navoda, ni ovi jedan zarez, ali ni ovi tacka-zarez. Zatim, kod pretvaranja RGB slike u monohromatsku sliku, ne sme da se zaboravi ključna rec `size`. Potom, dodela ovome `f` je obavezna. `f = double(f)` takodje se podrazumeva, jer kako bi se inace mogle raditi operacije nad slikom. E sad, vazno je ovo za masku da objasnimo. Ovde, kod maske, posto je potrebno da se detektuju vertikalne ivice, to onda znaci cela prva i druga kolona da su -1, pa onda sredisnja kolona 0, pa onda cetvrta i peta kolona da su 1. Zato sto su u pitanju vertikalne ivice. Da se traze horizontalne ivice, onda bi bilo prva i druga vrsta da su sve -1, pa onda treca vrsta sve 0, pa onda cetvrta i peta vrsta sve 1.

```
f = imread('slika5.jpg');
if size(f, 3) == 3
    f = rgb2gray(f);
end

f = double(f);

maska = [ -1, -1, 0, 1, 1 ; -1, -1, 0, 1, 1 ; -1, -1, 0, 1, 1 ; -1, -1, 0, 1, 1 ; -1, -1, 0, 1, 1 ];

slika_filtrirana = imfilter(f, maska, 'replicate') ;

subplot(1,2,1);
imshow(uint8(f));
title('Originalna slika');
subplot(1,2,2);
imshow(uint8(slika_filtrirana));
title('Vertikalne ivice');
```

b) Nad rezultatom dobijenim u prethodnom koraku izvršiti segmentaciju sa fiksnim pragom. Prag vrednosti izračunati kao srednju vrednost intenziteta svih piksela u slici. Rezultat segmentacije prikazati u novom prozoru. |

Resenje:

```
[m,n] = size(slika_filtrirana);  
  
suma = 0;  
  
for i = 1:1:m  
    for j = 1:1:n  
        suma = suma + slika_filtrirana(i, j);  
    end  
end  
  
T = suma / ( m * n);  
  
slika_segmentisana = slika_filtrirana > T;
```

Hajde da objasnimo o čemu se radi u ovom kodu. Dakle, `[m, n] = size ( slika_filtrirana )` je deo koda koji je obavezan, da bismo naglasili da je `m` broj vrsta, a `n` broj kolona. I naravno, da napomene po ko zna koji put, ključna rec `size` ni u ludilu ne sme da se zaboravi. Zatim, za sumu postavljamo inicijalnu vrednost 0. Zato što, naravno, od 0 počinjemo da računamo zbir svih ovih elemenata. Potom, pišemo ugnjezdenu petlju, prvo spoljasnju, pa potom i unutrašnju. Spoljasnja se odnosi na vrste matrice, a unutrašnja na kolone matrice. Promenljiva za vrste je `i`, a za kolone je `j`. Potom, pišemo `suma = suma + slika_filtrirana ( i, j );`. E sad, tu se vrti petlja i sabira element po element. Znači, sabere `0 + slika_filtrirana ( 1, 1 )` i to je prvi zbir, pa onda prvi zbir sabira sa elementom `slika_filtrirana ( 1, 2 )` i to je drugi zbir. Pa onda drugi zbir sabira sa elementom `slika_filtrirana ( 1, 3 )` i sve tako nadalje. Potom, ide `end` za kraj unutrašnje petlje, pa potom još jedan `end` za kraj spoljasnje petlje. Zatim, prag segmentacije označen sa `T` kažemo da je jednak količniku sume i dimenzije matrice, a dimenzija matrice se dobija kad se pomnoži broj vrsta * broj kolona. I onda je lako za kraj, `slika_segmentisana = slika_filtrirana > T`; `T` je više nego očigledno šta znači. To je to što se tiče objasnjenja ovoga.

3. Napisati Matlab program koji učitava sliku u boji (RGB) pod nazivom 'slika3.jpg' i izračunava normalizovani histogram nad G komponentom boje, bez korišćenja ugrađene Matlab funkcije f(). Na ekranu treba prikazati sliku i dobijeni normalizovani histogram.

```
% Učitavanje slike
f = imread('slika3.jpg');

% Izdvajanje G (zelene) komponente
G = f(:, :, 2);

% Inicijalizacija niza za histogram
histogram = zeros(1, 256); % po 1 element za svaki mogući intenzitet (0-255)

% Računanje histograma "ručno"
[m, n] = size(G);
for i = 1:m
    for j = 1:n
        vrednost = G(i, j);
        histogram(vrednost + 1) = histogram(vrednost + 1) + 1;
        % "+1" jer Matlab nema indeks 0
    end
end

% Normalizacija histograma
histogram_norm = histogram / (m * n);

% Prikaz slike i histograma
figure;
subplot(1, 2, 1);
```

1. ✓ Linija za G kanal mora biti

```
G = f(:, :, 2);
```

2. ✓ Pre petlji obavezno napravi histogram:

```
histogram = zeros(1, 256);
```

3. ✓ U petlji koristi:

```
vrednost = G(i, j);
```

```
histogram(vrednost+1) = histogram(vrednost+1) + 1;
```

4. ✓ Na kraju normalizacija:

```
histogram_normalizovani = histogram / (m*n);
```

5. ✓ I prikaz (ako hoćeš da ga vidiš):

```
subplot(1, 2, 1); imshow(uint8(G));
```

```
subplot(1, 2, 2); plot(histogram_normalizovani);
```

3. Napisati Matlab program koji učitava sliku u boji (RGB) pod nazivom 'slika3.jpg' i izračunava normalizovani histogram nad G komponentom boje, bez korišćenja ugrađene Matlab funkcije f(). Na ekranu treba prikazati sliku i dobijeni normalizovani histogram.

```
% Učitavanje slike
f = imread('slika3.jpg');

% Izdvajanje G (zelene) komponente
G = f(:, :, 2);

% Inicijalizacija niza za histogram
histogram = zeros(1, 256); % po 1 element za svaki mogući intenzitet (0-255)

% Računanje histograma "ručno"
[m, n] = size(G);
for i = 1:m
    for j = 1:n
        vrednost = G(i, j);
        histogram(vrednost + 1) = histogram(vrednost + 1) + 1;
        % "+1" jer Matlab nema indeks 0
    end
end

% Normalizacija histograma
histogram_norm = histogram / (m * n);

% Prikaz slike i histograma
figure;
subplot(1, 2, 1);
```

◆ 1 `G = f(:, :, 2);`

Tačno!

To znači da uzimaš zelenu komponentu (Green) iz RGB slike.

RGB ima 3 sloja:

- `f(:, :, 1)` — crvena,
- `f(:, :, 2)` — zelena,
- `f(:, :, 3)` — plava.

Zato mora da stoji `f(:, :, 2)` — jer `(:, :, 2)` bez `f` ne znači ništa.

2. `G = double(G);`

Opet tačno! Zato što je slika u tipu `uint8` (vrednosti 0–255), a kad radiš matematičke operacije, Matlab mora da ima realne brojeve. Inače bi pravio greške u sabiranju i deljenju.

3. Napisati Matlab program koji učitava sliku u boji (RGB) pod nazivom 'slika3.jpg' i izračunava normalizovani histogram nad G komponentom boje, bez korišćenja ugrađene Matlab funkcije f(). Na ekranu treba prikazati sliku i dobijeni normalizovani histogram.

```
% Učitavanje slike
f = imread('slika3.jpg');

% Izdvajanje G (zelene) komponente
G = f(:, :, 2);

% Inicijalizacija niza za histogram
histogram = zeros(1, 256); % po 1 element za svaki mogući intenzitet (0-255)

% Računanje histograma "ručno"
[m, n] = size(G);
for i = 1:m
    for j = 1:n
        vrednost = G(i, j);
        histogram(vrednost + 1) = histogram(vrednost + 1) + 1;
        % "+1" jer Matlab nema indeks 0
    end
end

% Normalizacija histograma
histogram_norm = histogram / (m * n);

% Prikaz slike i histograma
figure;
subplot(1, 2, 1);
```

◆ 3 `histogram = zeros(1, 256);`

Ovo pravi vektor od 256 nula, tj.

csharp

[0 0 0 0 ... 0]

svaka pozicija predstavlja broj piksela koji imaju taj intenzitet.

Na primer:

- `histogram(1)` broji piksele čiji je intenzitet 0
- `histogram(2)` broji piksele čiji je intenzitet 1
- ...
- `histogram(256)` broji piksele čiji je intenzitet 255

◆ 4 `vrednost = G(i, j);`

Tačno, to je trenutna vrednost intenziteta piksela.

3. Napisati Matlab program koji učitava sliku u boji (RGB) pod nazivom 'slika3.jpg' i izračunava normalizovani histogram nad G komponentom boje, bez korišćenja ugrađene Matlab funkcije f(). Na ekranu treba prikazati sliku i dobijeni normalizovani histogram.

```
% Učitavanje slike
f = imread('slika3.jpg');

% Izdvajanje G (zelene) komponente
G = f(:, :, 2);

% Inicijalizacija niza za histogram
histogram = zeros(1, 256); % po 1 element za svaki mogući intenzitet (0-255)

% Računanje histograma "ručno"
[m, n] = size(G);
for i = 1:m
    for j = 1:n
        vrednost = G(i, j);
        histogram(vrednost + 1) = histogram(vrednost + 1) + 1;
        % "+1" jer Matlab nema indeks 0
    end
end

% Normalizacija histograma
histogram_norm = histogram / (m * n);

% Prikaz slike i histograma
figure;
subplot(1, 2, 1);
```

5. $\text{histogram}(\text{vrednost} + 1) = \text{histogram}(\text{vrednost} + 1) + 1$; Ovo znači: ➡ „Idi na poziciju u histogramu koja odgovara tom intenzitetu i povećaj brojanje za 1.” Dakle: ako je piksel imao vrednost 45, ➡ $\text{histogram}(46)$ će se uvećati za 1. To se ponavlja za svaki piksel, i tako dobijaš ceo histogram slike.

6. Na kraju, $\text{histogram_normalizovani} = \text{histogram} / (m * n)$; To samo znači: ➡ podeli sve vrednosti sa ukupnim brojem piksela, da bi suma histograma bila 1. To je tzv. normalizovani histogram (svaka vrednost predstavlja verovatnoću pojavljivanja tog intenziteta).

1. Naredna matrica prikazuje blok monohromatske slike memorisan sa 6 bpp
- a) Izračunati i nacrtati normalizovani histogram ove slike.
- b) Piksele čija je verovatnoća pojavljivanja manja od 20% zameniti belim pikselima i prikazati matricu slike nakon ove operacije.
- c) Obaviti segmentaciju sa pragom $T=20$.

7	6	7
11	12	7
10	10	12
7	12	6

Imamo matricu slike:

```
[ 7,  6,  7 ;  
 11, 12,  7 ;  
 10, 10, 12 ;  
  7, 12,  6 ]
```

Koraci za računanje histograma (bez crtanja):

1. Odredimo sve moguće intenzitete piksela.

- Pošto je slika sa **6 bpp**, mogući pikseli su od 0 do $2^6 - 1 = 63$.
- U našoj slici se javljaju: 6, 7, 10, 11, 12.

2. Prebrojavanje pojava svakog intenziteta:

- Broj elemenata u slici: 4 reda $\times$ 3 kolone = 12 piksela.
- Koliko puta se pojavljuje svaki intenzitet:
 - ▶ 6 → pojavljuje se 2 puta
 - ▶ 7 → pojavljuje se 4 puta
 - ▶ 10 → pojavljuje se 2 puta
 - ▶ 11 → pojavljuje se 1 put
 - ▶ 12 → pojavljuje se 3 puta

3. Računanje verovatnoća (normalizovani histogram):

- Za svaki intenzitet i , vrednost histograma $h(i)$ je:

$$h(i) = \frac{\text{broj pojavljivanja } i}{\text{ukupan broj piksela}}$$

- $h(6) = 2/12 = 0.167$
- $h(7) = 4/12 = 0.333$
- $h(10) = 2/12 = 0.167$
- $h(11) = 1/12 = 0.083$
- $h(12) = 3/12 = 0.25$

4. Kako se histogram crta (tekstualno):

- Na **x-osi** stavljamo sve moguće intenzitete piksela (0–63), ili samo one koji se pojavljuju.
- Na **y-osi** stavljamo vrednosti histograma (broj pojavljivanja ili normalizovane verovatnoće).
- Za svaki intenzitet nacrtamo stub ili tačku čija visina odgovara $h(i)$.

Imamo prethodni normalizovani histogram:

Piksel	$h(i)$
6	0.167
7	0.333
10	0.167
11	0.083
12	0.25

Koraci za rešavanje zadatka:

1. Pronađemo piksele sa verovatnoćom manjoj od 20%:

- Prag $T = 0.2$ (20%)
- Pikseli čija je $h(i) < 0.2$:
 - ▶ $6 \rightarrow 0.167 < 0.2$ ✓ → menja se
 - ▶ $10 \rightarrow 0.167 < 0.2$ ✓ → menja se
 - ▶ $11 \rightarrow 0.083 < 0.2$ ✓ → menja se
- Pikseli koji ostaju isti:
 - ▶ $7 \rightarrow 0.333 \geq 0.2$
 - ▶ $12 \rightarrow 0.25 \geq 0.2$

2. Zamena odabranih piksela belom vrednošću:

- Pretpostavimo da bela boja u 6 bpp odgovara **maksimalnoj vrednosti 63** (jer 6 bpp $\rightarrow 0-63$).
- Svaki piksel koji se menja, postaje 63.

3. Originalna matrica slike:

```
[ 7,  6,  7 ;  
 11, 12,  7 ;  
 10, 10, 12 ;  
  7, 12,  6 ]
```

4. Primena zamene:

- Menjamo 6, 10, 11 u 63:

```
[ 7, 63,  7 ;  
 63, 12,  7 ;  
 63, 63, 12 ;  
  7, 12, 63 ]
```

Imamo istu matricu slike (6 bpp):

```
[ 7,  6,  7 ;  
 11, 12,  7 ;  
 10, 10, 12 ;  
  7, 12,  6 ]
```

Prag za segmentaciju: $T = 20$

1 Šta znači segmentacija sa pragom:

- Svaki piksel se poredi sa pragom T .
- Ako je piksel $\geq T$, postavlja se na **maksimalnu vrednost** (u 6 bpp $\rightarrow 63$, bela).
- Ako je piksel $< T$, postavlja se na 0 (crna).

Ovo je tzv. **binarna segmentacija slike**.

2 Provera naših piksela:

- Svi pikseli u matrici su: 6, 7, 10, 11, 12
- Prag $T = 20$
- Svaki piksel $< 20 \rightarrow$ postaje 0
- Svaki piksel $\geq 20 \rightarrow$ postaje 63

Sve vrednosti su < 20 , dakle svi pikseli postaju 0.

3 Rezultujuća matrica:

```
[ 0,  0,  0 ;  
 0,  0,  0 ;  
 0,  0,  0 ;  
 0,  0,  0 ]
```

2. Napisati Matlab program koji učitava monohromatsku sliku 'slika1.jpg' i obavlja linearno filtriranje slike sa maskom dimenzija 5×1 definisanom tako da računa srednju vrednost piksela ispod maske. Zatim nad dobijenim rezultatom filtriranja izvršiti segmentaciju sa fiksnim pragom $T = 10$. Prikazati originalnu sliku, kao i sliku nakon segmentacije.
-

♦ MATLAB kod (kompletan)

```
% 1. Učitavanje slike
slika = imread('slika1.jpg');

% Ako slika nije monohromatska,
konvertovati u grayscale
if size(slika, 3) == 3
    slika = rgb2gray(slika);
end

% 2. Definisanje maske 5x1 za srednju
vrednost
maska = ones(5, 1) / 5;

% 3. Linearno filtriranje (konvolucija)
slika_filtrirana = imfilter(slika, maska,
    'replicate');

% 4. Segmentacija sa fiksnim pragom T=10
T = 10;
slika_segmentisana = slika_filtrirana > T;
% binarna slika (1 = belo, 0 = crno)

% 5. Prikaz rezultata
figure;
subplot(1, 2, 1);
imshow(slika);
title('Originalna slika');

subplot(1, 2, 2);
imshow(slika_segmentisana);
title('Slika nakon segmentacije');
```

`slika = imread ( ' slika1.jpg ' );` // je učitavanje slike. To je prosto.

`if size(slika, 3) == 3` // znaci sledece: za monohromatsku sliku mi imamo 2 dimenzije, a to su x i y. Dakle visina i sirina. A za sliku u boji mi imamo R, G, B sto su 3 kanala boje, crvena, zelena, plava, i zato pise `size ( slika, 3 )` u uslovu. A ovo sto pise `== 3` je zbog toga sto proveravamo da li slika ima 3 kanala. Ako da, pretvaramo je u grayscale, tako sto napisemo `slika = rgb2gray ( slika )`; i posle end.

Maska je jasno kako se definise. ones u prevodu sa engleskog znaci jedinice, a ovo sto pise u zagradi su dimenzije matrice, ali tako da se dobija vektor. I onda se deli sa 5 prema uslovu zadatka.

Za linearno filtriranje je jasna naredba. Samo treba da nazovemo `slika_filtrirana` i onda napisemo `imfilter` i onda nadalje sve ide lako, samo navedemo naziv slike i masku i replicate i to je sve bukvalno.

Za segmentaciju znamo da je uslov ako je vece od ovog praga da ce biti 1 a ako je manje da ce biti nula i to je takodje intuitivno jasno. Samo voditi racuna, ovde treba da pise `slika_filtrirana > 10`, a ne `slika > 10`, tako je prema uslovu zadatka. Eto, to je sve manje vise.

3. Napisati postupak za linearno filtriranje slike $a(x,y)$ maskom $b(x,y)$ u frekvencijskom (Furijevom) domenu. *Napomena: ne treba pisati Matlab program.*
-

Postupak za linearno filtriranje slike $a(x, y)$ maskom $b(x, y)$ u frekvencijskom (Fourierovom) domenu:

1. Izračunavanje DFT slike

Računa se **diskretna Furijeova transformacija (DFT)** ulazne slike $a(x, y)$:

$$F(u, v) = \text{DFT}\{a(x, y)\}$$

2. Proširenje maske

Maska $b(x, y)$ (filter) se **proširi nulama** do dimenzija slike, da bi imale iste dimenzije.

Zatim se izračuna njena DFT:

$$H(u, v) = \text{DFT}\{b(x, y)\}$$

3. Množenje u frekvencijskom domenu

U frekvencijskom domenu se vrši **množenje po elementima**:

$$G(u, v) = F(u, v) \cdot H(u, v)$$

4. Izračunavanje inverzne DFT

Dobijeni rezultat $G(u, v)$ se vraća u prostorni domen pomoću **inverzne diskretne Furijeove transformacije (IDFT)**:

$$g(x, y) = \text{IDFT}\{G(u, v)\}$$

5. Uzimanje realnog dela rezultata

Pošto se pri transformaciji mogu javiti kompleksne vrednosti zbog numeričkih efekata, uzima se **realni deo** rezultujuće slike:

$$\text{Rezultat filtriranja} = \Re\{g(x, y)\}$$

 Konačno objašnjenje:

Ovim postupkom se filtriranje slike u prostornom domenu (konvolucija slike i maske) sprovodi **pomoću množenja u frekvencijskom domenu**, što je često efikasnije kod velikih slika.

4. Napisati Matlab program koji učitava kolor (RGB) sliku pod nazivom 'slika4.jpg' i izračunava kontrast samo unutar B kanala slike. Kontrast je definisan kao odnos maksimalnog i minimalnog intenziteta u slici. U zadatku nije dozvoljeno koristiti ugrađene Matlab funkcije `max()` i `min()`.

Ocu ja da probam da objasnim ovo da vidim kako mi ide. Prvo i prvo: u zadatku se traži da se izračuna kontrast samo unutar B kanala slike. Zato ide naredba `B = double(slika(:, :, 3));` // R i G ne znam šta se sa njima radi. I da, mora se pretvoriti u double da bi moglo da se izračuna kontrast. To je prvi deo zadatka, dalje ću ja da komentarisem kad mi kazes za ovo da li je dobro. Nemoj ti nista dalje da pricas, ocu ja malo da vidim kako shvatam stvari. Cujes.

`[m, n] = size(B);` a to su dimenzije B kanala. m je broj vrsta, a n broj kolona.

`maks = B(1, 1)`
`min = B(1,1)` // pretpostavljamo da je prvi element ono što tražimo, i upoređujemo sa svim ostalim. Bukvalno se setamo kroz petlju. For petlju.

E sad ovi endovi da iskomentarisem. Prvi end je za `for i = 1:m`, drugi end je za `for j = 1:n`, treci end je za `if B(i,j) > maks`, cetvrti end je za `if B(i,j) < min`. Eto tolko zasad.

- Kontrast u ovom zadatku je odnos **maksimalnog i minimalnog intenziteta**:
$$\text{kontrast} = \text{maks} / \text{min};$$
- To što si rekao da je kontrast "suprotnost" je u pravu u smislu da veći kontrast znači veće razlike između najsvetlijih i najtamnijih piksela.
- `disp(kontrast)` je baš naredba za prikaz vrednosti u komandnom prozoru – ne crta ništa, samo ispiše broj koji si izračunao.

Primer MATLAB koda (bez max/min funkcija):

```
% 1. Učitavanje slike
slika = imread('slika4.jpg');

% 2. Ekstrakcija B kanala
B = double(slika(:, :, 3)); %
konvertujemo u double da bi računali

% 3. Inicijalizacija
[m, n] = size(B);
maks = B(1,1);
min = B(1,1);

% 4. Pronalazak maks i min manuelno
for i = 1:m
    for j = 1:n
        if B(i,j) > maks
            maks = B(i,j);
        end
        if B(i,j) < min
            min = B(i,j);
        end
    end
end

% 5. Izračunavanje kontrasta
kontrast = maks / min;

% 6. Prikaz rezultata
disp(['Kontrast B kanala: ',
num2str(kontrast)]);
```

num2str(kontrast) je Matlab/Octave funkcija koja pretvara broj u string (tekst).

Zašto je to korisno:

- Naredba disp() može direktno prikazati broj, ali ako hoćeš da **ispiseš poruku zajedno sa brojem**, moraš da spojiš tekst i broj.

- Primer:

```
disp(['Kontrast B kanala je: '
num2str(kontrast)]);
```

- Ovde:

- 'Kontrast B kanala je: ' je tekstualni string.
- num2str(kontrast) pretvara broj kontrasta u string.
- disp() onda ispiše celokupan tekst sa brojem.

Bez num2str, ne bi mogao direktno spojiti tekst i broj u istom ispisku.

U RGB slici:

- Prvi kanal (1) je **R** (crvena),
- Drugi kanal (2) je **G** (zelena),
- Treći kanal (3) je **B** (plava).

Zato kad napišeš:

```
B = slika(:, :, 3);
```

ti uzimaš **treći kanal**, tj. plavu komponentu slike.

Nema veze sa redom u slici, nego baš redosledom kanala u RGB reprezentaciji slike.

U Matlab-u, kada učitáš sliku preko `imread`, vrednosti piksela su obično **tipa uint8** (unsigned 8-bit integer). To znači da su celobrojne vrednosti između 0 i 255.

Problem je što **matematičke operacije kao što su deljenje, računanje proseka, kontrasta ili normalizacija ne rade "precizno" na uint8** — npr. $100/200$ bi bilo 0 jer je integer deljenje.

Zato se koristi:

```
B = double(slika(:, :, 3));
```

- `double` pretvara vrednosti piksela u **brojeve sa pokretnim zarezmom** (decimalne vrednosti), npr. 100 postaje 100.0.
- Sada možeš računati maksimum, minimum, odnos kontrasta i sve druge formule bez problema.

2. Neka su zadati nizovi $x = [7, 2, 10, 8, 11, 6, 3]$ i $y = [10, 1, 2, 4, 8, 10, 8]$.

a) Odrediti vrednosti median-a ovih nizova.

b) Koju vrstu filtriranja predstavlja median filtriranje slike i koja je njegova namena?

c) Odrediti Euklidsko rastojanje ova dva niza.

Vrednosti mediana ovih nizova se dobijaju tako sto kod oba niza uradimo istu stvar. Poredjamo elemente od najmanjeg do najveceg, pa kad to uradimo onda jednostavno uzmemo sredisnji element. To ako je neparan broj elemenata niza, a ako je paran, onda saberemo 2 sredisnja elementa i izdelimo sa 2.

Median filtriranje slike je nelinearno filtriranje. Namena: Koristi se za uklanjanje šuma sa slike, naročito impulsnog šuma (tzv. "salt-and-pepper" šum). Očuva ivice slike bolje nego linearni filteri (npr. srednja vrednost), jer ne računa prosek već središnju vrednost (medijan) piksela u zadanom prozoru. Kako funkcioniše: Za svaki piksel slike definiše se mali kvadratni ili pravougaoni prozor oko njega (npr. 3×3 , 5×5). Pikseli unutar tog prozora se sortiraju po intenzitetu. Originalni piksel se zameni medijanom tog niza vrednosti. Rezultat: šum se značajno smanjuje, a ivice i detalji se zadržavaju.

■ Euklidsko rastojanje između dva niza (ili vektora) meri koliko su "udaljeni" jedan od drugog u prostoru. Ako imamo dva niza (ili tačke)

$$A = (a_1, a_2, a_3, \dots, a_n) \quad B = (b_1, b_2, b_3, \dots, b_n),$$

$$d(A, B) = \sqrt{(a_1 - b_1)^2 + (a_2 - b_2)^2 + (a_3 - b_3)^2 + \dots + (a_n - b_n)^2}$$

📖 Dakle, kao što si rekao:

1. Uzimamo razliku elemenata **na istim pozicijama**.
2. **Kvadriramo** svaku razliku.
3. Sve to **saberemo**.
4. Na kraju stavimo **veliki kvadratni koren** preko tog zbira.

3. Ako matricu iz Zadatka 1 označimo sa f , odrediti rezultate izvršenja sledećih Matlab naredbi: a. $f(:, 2:4)$ b. $f > 5$ c. $\min(f)$ d. $f(2:4, 2:4) * f(2:4, 1:3)$

$$f = \begin{bmatrix} 2 & 1 & 7 & 6 & 7 \\ 1 & 3 & 3 & 5 & 7 \\ 3 & 2 & 3 & 6 & 3 \\ 6 & 1 & 7 & 7 & 1 \end{bmatrix}$$

a) $f(:, 2:4)$

Znači: sve vrste (:) ali samo kolone 2 do 4.
Dakle, uzimamo 2., 3. i 4. kolonu.

Rezultat: $\begin{bmatrix} 1 & 7 & 6 \\ 3 & 3 & 5 \\ 2 & 3 & 6 \\ 1 & 7 & 7 \end{bmatrix}$

b) $f > 5$

Ovo vraća logičku (boolean) matricu iste dimenzije kao f , gde je svaki element 1 (true) ako je veći od 5, a 0 (false) ako nije.

c) $\min(f)$

Funkcija $\min(f)$ bez drugog argumenta uzima minimum svake kolone (po vrstama). Rezultat: $[1, 1, 3, 5, 1]$

d) $f(2:4, 2:4) * f(2:4, 1:3)$

Ovo je pravo množenje matrica (ne element po element). Prvo izdvojimo podmatrice:

$$f(2:4, 2:4) = \begin{bmatrix} 3 & 3 & 5 \\ 2 & 3 & 6 \\ 1 & 7 & 7 \end{bmatrix}, \quad f(2:4, 1:3) = \begin{bmatrix} 1 & 3 & 3 \\ 3 & 2 & 3 \\ 6 & 1 & 7 \end{bmatrix}$$

Zatim se one množe:

$$\begin{bmatrix} 3 & 3 & 5 \\ 2 & 3 & 6 \\ 1 & 7 & 7 \end{bmatrix} \times \begin{bmatrix} 1 & 3 & 3 \\ 3 & 2 & 3 \\ 6 & 1 & 7 \end{bmatrix} = \begin{bmatrix} 51 & 20 & 65 \\ 48 & 17 & 57 \\ 67 & 26 & 73 \end{bmatrix}$$

4. Napisati Matlab program koji učitava monohromatsku sliku '*zad4.png*' i obavlja filtriranje slike u prostornom domenu. Masku filtra definisati tako da su joj dimenzije 3x3 i da izračunava razliku piksela ispod centralne vrste i centralne kolone maske. Na ekranu prikazati originalnu sliku, i sliku nakon filtriranja.
-

```
clc;
clear all;
close all;

% 1. Ucitavanje slike
slika = imread('zad4.png');

% 2. Provera da li je slika u RGB formatu
if size(slika, 3) == 3
    slika = rgb2gray(slika); % prebacivanje u monohromatsku (sivu)
end
slika = double(slika); % radi lakšeg računanja

% 3. Definisanje maske (3x3)
% maska računa razliku između piksela ispod centralne vrste i centralne kolone
maska = [0 -1 0; 1 0 1; 0 -1 0];

% 4. Filtriranje slike u prostornom domenu
slika_filtrirana = imfilter(slika, maska, 'replicate');

% 5. Prikaz originalne i filtrirane slike
figure;
subplot(1,2,1);
imshow(uint8(slika));
title('Originalna slika');

subplot(1,2,2);
imshow(uint8(slika_filtrirana));
title('Filtrirana slika');
```

1. Jedan blok monohromatske slik edimenzija 3x3 prikazan je sledećom matricom

a) Prikazati vrednosti piksela slike nakon postupka linearnog širenja kontrasta, pri čemu se opseg ulaznih vrednosti [10, 30] linearno proširuje na opseg [5, 50].

25	15	15
15	10	25
15	10	25

Određivanje $s=F(r)$? Jednačina prave kroz dve tačke:

koordinate 1. tačke: $(r_{min}, 0)$

koordinate 2. tačke: (r_{max}, s_{max})

$$\frac{x - x_1}{x_2 - x_1} = \frac{y - y_1}{y_2 - y_1} \quad \frac{r - r_{min}}{r_{max} - r_{min}} = \frac{s - 0}{s_{max} - 0}$$

$$s = \frac{s_{max}}{r_{max} - r_{min}} r - \frac{s_{max} r_{min}}{r_{max} - r_{min}}$$

Imamo jedan blok (deo slike) koji je monohromatski (tj. crno-beli), i dat je kao matrica od 3×3 piksela:

$$f = \begin{bmatrix} 25 & 15 & 15 \\ 15 & 10 & 25 \\ 15 & 10 & 25 \end{bmatrix}$$

Treba da se uradi **linearno širenje kontrasta**, tj. da se „raširi“ dinamički opseg intenziteta piksela.

Zadato je:

- Ulazni opseg vrednosti piksela:

$$[r_{min}, r_{max}] = [10, 30]$$

- Novi, prošireni opseg:

$$[s_{min}, s_{max}] = [5, 50]$$

♦ Šta znači linearno širenje kontrasta?

Zamisli da trenutno svi pikseli slike imaju vrednosti između **10 i 30**.

To je prilično „usk“ raspon (razlika 20), pa slika izgleda **isprano, bez kontrasta**.

Hoćemo da te vrednosti **rastegnemo** tako da sada zauzimaju ceo novi raspon od **5 do 50** — to jest, da tamni tonovi postanu još tamniji, a svetli još svetliji.

♦ Formula za linearno širenje kontrasta

Za svaki piksel slike koji ima originalnu vrednost r nova vrednost s računa se ovako:

$$s = s_{min} + \frac{(r - r_{min})(s_{max} - s_{min})}{r_{max} - r_{min}}$$

To je **linearno preslikavanje** jednog intervala u drugi.

♦ Uvrštavamo konkretne brojeve

$$r_{min} = 10, \quad r_{max} = 30$$

$$s_{min} = 5, \quad s_{max} = 50$$

Sad izračunamo faktor:

$$= \frac{s_{max} - s_{min}}{r_{max} - r_{min}} = \frac{50 - 5}{30 - 10} = \frac{45}{20} = 2.25$$

Dakle, formula postaje: $s = 5 + (r - 10) \cdot 2.25$
ili uprošćeno: $s = 2.25r - 17.5$

b) Koja je namena postupka širenja kontrasta? Navesti još neki postupak slične namene.

Namena postupka širenja kontrasta:

- Povećava vizuelni kontrast slike tako što proširuje opseg intenziteta piksela.
- Tamni delovi slike postaju tamniji, a svetli svetliji, što pomaže u isticanju detalja koji bi inače bili teško vidljivi.
- Koristi se za poboljšanje kvaliteta slike za ljudsko oko ili za dalje automatske obrade slike (npr. segmentaciju ili prepoznavanje objekata).

Primer sličnog postupka:

- **Histogram ekvilizacija** – raspoređuje intenzitete piksela ravnomernije preko celog opsega, čime se takođe poboljšava kontrast, ali ne linearno, već adaptivno prema raspodeli piksela.

2. Prikazati izgled negative slike iz zadatka 1, uz pretpostavku da je slika memorisana sa 6 bita po pikselu.

Razumem, zapisao. ✓

Napomena je bitna: pošto je slika 6 bita po pikselu, to znači da pikseli mogu imati vrednosti od 0 do 63 (jer $2^6 - 1 = 63$). To je važno kada se radi negativ slike, jer se formula za negativ glasi:

$$\text{negativ} = L - 1 - f(x, y)$$

gde je L broj nivoa (ovde 64), a $f(x, y)$ vrednost piksela.

Sve ostalo oko crtanja ili računanja ostaje isto.

a) Napisati konvolucionu teoremu za 2D digitalne slike i objasniti njene primene u digitalnoj obradi slike.

Konvoluciona teorema za 2D digitalne slike

Za dve 2D diskretne funkcije (slike) $f(x, y)$ i $h(x, y)$ važi: $\mathcal{F}\{f(x, y) * h(x, y)\} = \mathcal{F}\{f(x, y)\} \cdot \mathcal{F}\{h(x, y)\}$ i obrnuto:

$$f(x, y) * h(x, y) = \mathcal{F}^{-1}\{\mathcal{F}\{f(x, y)\} \cdot \mathcal{F}\{h(x, y)\}\}$$

Gde:

- $*$ označava 2D konvoluciju u prostornom domenu,
- $\mathcal{F}$ označava 2D Diskretnu Furijeovu transformaciju (DFT),
- $\cdot$ označava element-po-element množenje u frekvencijskom domenu,
- $\mathcal{F}^{-1}$ je inverzna DFT.

Objašnjenje

1. Konvolucija u prostornom domenu (klasični filter preko maske) može da bude računato i u frekvencijskom domenu.
2. Umesto direktnog sabiranja i množenja piksela kroz masku (što je sporo kod velikih slika i maski), možemo:
 - Izračunati DFT slike i DFT maske,
 - Pomnožiti ih element-po-element u frekvencijskom domenu,
 - Izračunati inverznu DFT i dobiti rezultat konvolucije.

Ovo je osnova brze obrada slika preko FFT algoritma.

b) Ako su u prostornom domenu zadati slika $a(x,y)$ i maska $b(x,y)$, napisati postupak za linearno filtriranje slike u frekvencijskom (Furijevom) domenu.

Prvi korak je **zero-padding** — tačno si rekao i način proširenja:

▴ nova dimenzija slike i maske treba da bude: $(M + P - 1) \times (N + Q - 1)$

Dakle — **tačno**, to je **prvi korak** u postupku filtriranja u frekvencijskom domenu.

Drugi korak je da se nad proširenim matricama $a(x,y)$ i $b(x,y)$ primeni **2D Furijeova transformacija**, čime se dobijaju: $A(u,v) = \text{FFT}\{a(x,y)\}$, $B(u,v) = \text{FFT}\{b(x,y)\}$

Dakle — **potpuno tačno**, to je **drugi korak**.

Treći korak je **element-po-element množenje** Furijevih transformacija slike i maske:

$$C(u,v) = A(u,v) \cdot B(u,v)$$

U MATLAB-u se to radi sa operatorom `.*` za **po-elementno množenje**, što odgovara tvojoj formulaciji.

Četvrti korak je **inverzna Furijeova transformacija** nad matricom C , tj. nad rezultatom element-po-element množenja.

Dakle: $g(x,y) = \text{ifft2}(C)$

Posle toga **realni deo** dobijene matrice predstavlja filtriranu sliku.

4. Napisati Matlab program koji izračunava minimalni element matrice zadate kao $A = \begin{bmatrix} 8, & 3, & 4, & 1; \\ 5, & 2, & 3, & 7; \\ 3, & 5, & 9, & 1 \end{bmatrix}$, bez korišćenja Matlab funkcije min().

```
% Definisanje matrice
```

```
A = [8, 3, 4, 1;  
     5, 2, 3, 7;  
     3, 5, 9, 1];
```

```
% Postavljanje prvog elementa kao početni minimum
```

```
minElem = A(1,1);
```

```
% Dobijanje dimenzija matrice
```

```
[m, n] = size(A);
```

```
% Ugnježdene petlje kroz vrste i kolone
```

```
for i = 1:m
```

```
    for j = 1:n
```

```
        if A(i,j) < minElem
```

```
            minElem = A(i,j); % ažuriranje minimuma
```

```
        end
```

```
    end
```

```
end
```

```
% Prikaz rezultata
```

```
disp(['Minimalni element matrice je: ', num2str(minElem)]);
```

1. Postavi početni minimum na prvi element matrice: `A(1,1)`.

2. Spoljašnja petlja ide kroz sve vrste (redove).

3. Unutrašnja petlja ide kroz sve kolone te vrste.

4. Upoređivanje: svaki element matrice se upoređuje sa trenutnim minimumom. Ako je manji, taj element postaje novi minimum.

5. Na kraju, nakon što sve elemente prođeš, u promenljivoj za minimum imaš najmanji element matrice.

I sve to bez korišćenja ugrađene funkcije `min()`.

b) Koja je namena postupka širenja kontrasta? Navesti još neki postupak slične namene.

Namena postupka širenja kontrasta:

- Povećava vizuelni kontrast slike tako što proširuje opseg intenziteta piksela.
- Tamni delovi slike postaju tamniji, a svetli svetliji, što pomaže u isticanju detalja koji bi inače bili teško vidljivi.
- Koristi se za poboljšanje kvaliteta slike za ljudsko oko ili za dalje automatske obrade slike (npr. segmentaciju ili prepoznavanje objekata).

Primer sličnog postupka:

- **Histogram ekvilizacija** – raspoređuje intenzitete piksela ravnomernije preko celog opsega, čime se takođe poboljšava kontrast, ali ne linearno, već adaptivno prema raspodeli piksela.

2. Napisati Matlab program koji učitava kolor (RGB) sliku pod nazivom 'test.jpg' I detektuje horizontalne ivice samo nad G komponentom slike. Za detekciju horizontalnih ivica definisati odgovarajuću masku dimenzija 5×3 . Na ekranu prikazati samo G komponentu slike nakon izdvajanja ivica.

```
f = imread('test.jpg');
slika = double(f);
G = slika(:, :, 2);
maska = [ -1, -1, -1;
          -1, -1, -1;
           0,  0,  0;
           1,  1,  1;
           1,  1,  1 ];

G_izdvojeno = imfilter(G, maska, 'replicate');

subplot(1, 2, 1);
imshow(G, []);
title('Originalna G komponenta');

subplot(1, 2, 2);
imshow(G_izdvojeno, []);
title('Horizontalne ivice');
```

3. Napisati Matlab program koji učitava sliku u boji (RGB) pod nazivom 'slika3.jpg' i izračunava normalizovani histogram nad G komponentom boje, bez korišćenja ugrađene Matlab funkcije `fft`. Na ekranu treba prikazati sliku i dobijeni normalizovani histogram.
-

```
f = imread('slika3.jpg');      % Učitavanje slike
G = f(:, :, 2);               % G komponenta
[m, n] = size(G);             % Dimenzije G komponente

% Inicijalizacija histograma od 0 do 255
histogram_G_manual = zeros(1, 256);

% Brojanje pojavljivanja svakog intenziteta
for i = 1:m
    for j = 1:n
        intenzitet = G(i,j);
        histogram_G_manual(intenzitet + 1) = histogram_G_manual(intenzitet + 1) + 1;
    end
end

% Normalizacija
normalizovani_histogram = histogram_G_manual / (m * n);

% Prikaz rezultata
subplot(1,2,1);
imshow(f);
title('Originalna slika');

subplot(1,2,2);
```



4. Napisati Matlab program koji učitava kolor sliku (RGB) pod nazivom 'slika4.jpg' i na osnovu nje formira monohromatsku sliku tj. matricu u kojoj su intenziteti piksela dobijeni kao srednja vrednost R, G i B komponenti
kolor
slike.
Nad dobijenom monohromatskom slikom izračunati njen negativ i prikazati rezultat na ekranu.
-

```
f = imread ( ' slika4.jpg ' );  
  
% Pretvaranje u grayscale (ako je RGB)  
if size(f,3) == 3  
    f_gray = rgb2gray(f);  
else  
    f_gray = f; % već monohromatska slika  
end  
  
% Pretvaranje u double radi daljih operacija  
f_gray = double(f_gray);  
  
% Negativ slike  
f_negativ = 255 - f_gray;  
  
% Prikaz originalne i negativ slike  
subplot(1,2,1);  
imshow(uint8(f_gray));  
title('Originalna monohromatska slika');  
  
subplot(1,2,2);  
imshow(uint8(f_negativ));  
title('Negativ slike');
```